FOUNDATIONS OF WEB APP PENTESTING

11.1 CSRF OVERVIEW



HAROON SHAHID

FOUNDATIONS OF WEB APP PENTESTING

11.1 CROSS-SITE REQUEST FORGERY (CSRF) OVERVIEW

==========

CSRF vs. XSS

I'd define them around **what the attacker controls** and **where the attacker's code executes**.

CSRF definition

Cross-Site Request Forgery (CSRF) is a web vulnerability where an attacker tricks a victim's browser into sending an unwanted request to a vulnerable website, causing the website to perform an action on the victim's behalf. In traditional CSRF, the attack abuses the victim's existing authentication, such as session cookies, to make the forged request appear legitimate. The attacker does not need to execute code inside the target website. Although CSRF commonly exploits an existing authenticated session, some variants, such as login CSRF, can target authentication workflows even when the victim is not previously logged in.

The distinction from XSS

The easiest way to separate them is:

> **CSRF:** The attacker makes the victim's browser **send a request to the target website**.
>
> **XSS:** The attacker gets **their own JavaScript to execute inside the target website's security context/origin**.

Or even more simply:

CSRF → "Make the victim's browser DO something."

XSS → "Make the victim's browser RUN my JavaScript."

The critical difference

Imagine the victim is logged into: **bank.com**

With CSRF:

attacker.com

↓

tricks victim's browser

↓

POST bank.com/change-email

↓

victim's bank cookie is included (if browser cookie rules permit it)

↓

bank changes the email (if the request is accepted without adequate CSRF protection)

The attacker's JavaScript **never becomes JavaScript running inside bank.com** in this traditional CSRF example.

With XSS:

attacker-controlled input

↓

bank.com

↓

vulnerability causes it to execute

↓

attacker's JavaScript runs in the security context of bank.com

That's the fundamental distinction.

One correction to an idea that can cause confusion

Don't define CSRF as:

> "An attacker sends a request from another website."

That's **too broad**. Cross-origin requests happen normally on the web.

The important part is:

The attacker causes the victim's browser to make an unwanted request to the target, and the target accepts it as an action performed under the victim's authority.

And don't define XSS simply as:

> "An attacker executes JavaScript."

That's also incomplete. The important part is that the attacker's script executes **in the security context/origin of the vulnerable target website**.

So if you want the **best two definitions to put side-by-side**, I'd use:

CSRF: An attack where an attacker tricks a victim's browser into sending an unwanted request to a website, causing that website to perform an action under the victim's authority, commonly through an existing authenticated session.

XSS: A vulnerability where an attacker-controlled script is executed in a victim's browser within the security context of a vulnerable website, allowing the script to act as that website's client-side code.

Traditional CSRF abuses the browser's automatic authentication behavior.

XSS abuses the website's ability to execute attacker-controlled code.

That's the distinction you want burned into your head.

Important: XSS can undermine CSRF protections

CSRF and XSS are different vulnerabilities, but they are not mutually exclusive.

If an attacker can execute JavaScript inside the target application's origin through XSS, that JavaScript can often:

- Read CSRF tokens accessible to the application's JavaScript.
- Submit authenticated requests.
- Include valid CSRF tokens in those requests.
- Perform sensitive operations using the victim's session.

Therefore, **CSRF tokens do not reliably protect against an attacker who already has JavaScript execution within the application's trusted security context**.

This is one reason XSS can be more powerful than traditional CSRF.

==========

The Core Mechanism: Exploiting Ambient Authentication

Browsers automatically attach applicable cookies, including session cookies, to HTTP requests when the cookie's scope, security attributes, SameSite restrictions, and browser policies permit it.

CSRF exploits this automatic authentication behavior when an attacker can cause a victim's browser to send an authenticated request that the target application accepts without adequate CSRF protection.

How the attack works

1. **The Setup:** The victim is logged into **bank.com**. Their browser holds a session cookie for **bank.com**.
2. **The Trap:** The victim visits **attacker.com** in another tab.
3. **The Attack:** **attacker.com** contains a hidden form or script that sends a **POST** request to **bank.com/transfer-funds**.
4. **The Result:** If browser cookie rules permit it, the browser attaches the victim's **bank.com** session cookie to that request. If the bank lacks effective CSRF defenses and the request meets the endpoint's requirements, the bank processes the transfer as an authenticated action.

Important: Browsers do not attach every cookie to every request.

Whether a cookie is included depends on attributes and policies such as:

- **Domain**
- **Path**
- **Secure**
- **SameSite**
- Cookie expiration
- Browser privacy and third-party cookie restrictions

Also, authentication is not limited to cookies. Other automatically supplied credentials, such as certain browser-managed HTTP authentication credentials, can also create CSRF risks.

==========

Characteristics of CSRF

Property	Description
Blind Attack	Traditional CSRF is generally a blind attack. An attacker can cause a victim's browser to send an authenticated request to a target application, but normally cannot read the response from an unrelated origin because of browser security restrictions such as the Same-Origin Policy. Exceptions and indirect information leaks may exist.
State-Changing	CSRF primarily targets actions performed under a victim's authority, such as changing account settings, transferring funds, or modifying application data. Although traditional CSRF does not normally let the attacker directly read protected responses, it may indirectly expose sensitive information depending on the application's functionality.
User Interaction	CSRF requires an attacker to cause the victim's browser to issue a forged request. This is commonly achieved through a malicious webpage, link, HTML form, or other attacker-controlled content that can initiate browser requests. The exact interaction required depends on the attack vector.

Important: Same-Origin Policy does not prevent all cross-origin requests

This distinction is fundamental to understanding CSRF.

A malicious website can generally cause a victim's browser to submit a normal HTML form to another origin.

The **Same-Origin Policy (SOP)** generally prevents attacker-controlled JavaScript from reading the resulting protected cross-origin response, not from causing ordinary browser-supported cross-origin requests to be sent.

Remember:

Sending a cross-origin request ≠ Reading a cross-origin response.

This is why CSRF can exist even when the application's CORS configuration does not allow the attacker to read responses.

==========

Impact of Successful Attacks

Account Takeover (ATO)

By forcing an email update or exploiting a vulnerable password-changing workflow, an attacker may be able to take control of an account.

However, account takeover depends on the application's functionality.

For example, forcing an email change does not automatically guarantee account takeover unless the attacker can subsequently exploit that change, such as through a password-reset workflow.

Business Logic Abuse

Forcing users to perform actions like "Liking" a post, following an account, or sending spam invitations.

Financial Loss

Unauthorized fund transfers or fraudulent purchases.

Reputation Damage

Posting inflammatory content on a user's behalf on social media.

Indirect Information Disclosure

An attacker may be able to redirect future sensitive communications or information to an attacker-controlled destination by modifying account settings.

Privilege or Permission Changes

An attacker may force an authenticated, sufficiently privileged user to modify permissions, add users, or change security settings when the application permits these actions without effective CSRF protection.

==========

Defenses and Prevention

To stop CSRF, the application needs effective checks that prevent attacker-induced requests from being accepted as authorized actions merely because the browser supplies valid authentication credentials.

CSRF defenses do not necessarily prove that a user consciously intended an action. Instead, they establish conditions that make unauthorized cross-site requests difficult or impossible to forge.

Anti-CSRF Tokens (Synchronizer Token Pattern)

The server generates a unique, cryptographically strong, unpredictable token for the user session.

- This token is embedded in forms, usually as a hidden input field, or delivered through another protected application response.
- The browser submits the token with the sensitive request.
- The server validates the submitted token against the value associated with the appropriate session.
- The server rejects protected state-changing operations when the token is missing or invalid.

Important: This is the synchronizer token pattern. Other valid token mechanisms, such as signed double-submit cookies, do not necessarily store a copy of each token in the server session.

A token should not be leaked through URLs, logs, or other locations accessible to attackers.

SameSite Cookie Attribute

- **Strict:** Cookies are sent in same-site requests, but not in cross-site requests. This includes ordinary navigations from unrelated sites.

- **Lax:** Cookies are sent for same-site requests and for cross-site top-level navigations using safe HTTP methods, typically **GET**. They are generally not sent with cross-site POST requests, images, iframes, or other subresource requests.
- **None:** Removes SameSite restrictions. Modern browsers require **Secure** when using **SameSite=None**. Other browser policies, including third-party cookie restrictions, can still prevent transmission.

Important exception: Lax-by-default

When the **SameSite** attribute is omitted, some browsers apply a default Lax policy.

Certain implementations use a more permissive default that allows recently created cookies to accompany cross-site top-level POST requests for approximately two minutes.

This exception does not apply in the same way to cookies explicitly configured with **SameSite=Lax**.

Never assume that omitting SameSite and explicitly setting **SameSite=Lax** have identical behavior.

Origin and Referer Validation

The server can examine:

- **Origin**
- **Referer**

These headers can help determine where a request originated.

For example: **Origin: https://attacker.com**

If the sensitive endpoint belongs to **https://bank.com**, the server can reject requests originating from untrusted origins.

Important: Validation must compare trusted origins correctly. Missing headers, **Origin: null**, redirects, and improperly implemented validation rules require careful handling.

A missing Origin or Referer header should not automatically be treated as evidence that the request is legitimate.

Fetch Metadata Headers

Modern browsers can include headers such as: **Sec-Fetch-Site: cross-site**

The server can use Fetch Metadata to determine the request's relationship to the destination.

Common values include:

- **same-origin**
- **same-site**

- `cross-site`
- `none`

An application can reject suspicious cross-site state-changing requests according to its security policy.

However, Fetch Metadata implementations need correctly configured exceptions and fallback protections for requests where these headers are unavailable.

Important: `same-site` does not mean `same-origin`. An attacker-controlled sibling subdomain may still generate requests classified as same-site.

Custom Request Headers

Some APIs require a custom HTTP header on sensitive requests.

For example: `X-CSRF-Token: abc123`

A cross-origin JavaScript request containing a non-safelisted custom header normally requires a successful CORS preflight.

A standard HTML form cannot simply attach arbitrary custom HTTP headers.

Therefore, requiring an appropriate custom header, combined with a restrictive CORS policy and correct server-side validation, can provide CSRF protection.

However, this protection can fail if the CORS policy permits untrusted origins or another application weakness allows the attacker to supply the required header.

Re-authentication

Requiring the user to provide their current password or a fresh MFA verification before high-value actions, such as password changes, can prevent automated CSRF attacks from completing those actions.

This protection depends on the application securely validating the fresh authentication requirement.

Re-authentication should be considered an additional security layer rather than a universal replacement for CSRF protection.

Important: CSRF protection is not authorization

A valid CSRF token does not prove that the user has permission to perform an action.

Applications must independently enforce:

- Authentication

- Authorization
- Appropriate CSRF protection for vulnerable workflows

These controls solve different security problems.

Question: Why Do We Need CSRF Tokens if There Is a SameSite Flag?

SameSite cookies can prevent many CSRF attacks, so you might reasonably ask: *why do we still need CSRF tokens?*

The key is:

> **SameSite controls whether the browser includes cookies in a request based on its site context. CSRF tokens allow the server to verify that a request contains a valid, unpredictable value associated with the appropriate session or otherwise securely validated. This makes it difficult for an external attacker to forge an acceptable request.**

Let's use real web requests rather than analogies.

First, what is the actual CSRF problem?

Suppose you're logged into: **https://bank.com**

Your browser has: **Cookie: session=ABC123**

Now you visit: **https://evil.com**

The attacker has this HTML:

```
<form action="https://bank.com/change-email" method="POST">
  <input name="email" value="attacker@gmail.com">
</form>

<script>
  document.forms[0].submit();
</script>
```

For this example, assume:

1. The bank's session cookie is permitted in the cross-site request, for example, it uses **SameSite=None; Secure** and is not blocked by another browser policy.
2. The application does not implement effective CSRF protection.
3. The **/change-email** endpoint accepts this type of form submission.

Under those conditions, your browser sends a request similar to:

```
POST /change-email HTTP/1.1
Host: bank.com
Cookie: session=ABC123
Content-Type: application/x-www-form-urlencoded

email=attacker%40gmail.com
```

The bank sees:

```
session=ABC123
email=attacker@gmail.com
```

It treats the request as an authenticated operation.

So your email gets changed.

That's traditional CSRF.

Where does SameSite help?

The bank can set:

```
Set-Cookie: session=ABC123; Path=/; Secure; HttpOnly; SameSite=Lax
```

Now the browser has a rule:

> Don't send this cookie in cross-site requests unless they satisfy the specific conditions permitted by Lax.

So when **evil.com** tries to submit the cross-site POST request to: **https://bank.com/change-email**

the browser does **not normally send** the explicitly configured **SameSite=Lax** session cookie.

The bank receives a request resembling:

```
POST /change-email HTTP/1.1
Host: bank.com
Content-Type: application/x-www-form-urlencoded

email=attacker%40gmail.com
```

The session cookie is absent.

If the endpoint requires that cookie for authentication, the attack fails.

So yes, **SameSite is genuinely a CSRF defense.**

Then why CSRF tokens?

Because SameSite is not the same type of protection.

Imagine the bank uses: **SameSite=None; Secure**

Now SameSite itself does not prevent the cookie from being sent cross-site.

If browser cookie policies permit the request and the application lacks other protections, a CSRF attack may work.

With a CSRF token, the legitimate form might look like:

```
<form action="/change-email" method="POST">
  <input name="email" value="new@email.com">
  <input
    type="hidden"
    name="csrf_token"
    value="8f91a72c..."
  >
</form>
```

The legitimate request becomes:

```
POST /change-email HTTP/1.1
Host: bank.com
Cookie: session=ABC123
Content-Type: application/x-www-form-urlencoded

email=new%40email.com&csrf_token=8f91a72c...
```

The server checks:

Is csrf_token valid for this session or otherwise valid under the application's securely implemented CSRF-token scheme?

If yes, and the user is authorized and all other requirements are satisfied: **Change the email.**

If an attacker causes a request without a valid token:

```
POST /change-email HTTP/1.1
Host: bank.com
Cookie: session=ABC123
Content-Type: application/x-www-form-urlencoded

email=attacker%40gmail.com
```

The server rejects the operation.

The response may be: **403 Forbidden**

But different applications may return `400`, a redirect, or another response.

The important result is that the protected operation must not succeed.

But here's the important part

You might say:

> "Fine. Just use `SameSite=Strict` everywhere."

That **can work extremely well** against traditional cross-site requests in cookie-based applications.

But there are situations where you **can't simply rely on SameSite**.

For example, suppose an application legitimately needs: `SameSite=None`

because its authentication/session cookie must work in a cross-site context.

Now SameSite cannot restrict the cookie in that context.

A properly implemented CSRF token still can protect the sensitive operation.

Even `SameSite=Strict` is not a universal defense against attacks that originate from the same site, such as certain attacks involving compromised sibling subdomains or vulnerable application-side JavaScript.

Another important difference: SameSite doesn't understand your application

Suppose your legitimate application has:

```
POST /transfer-money
with amount=500 to=12345
```

SameSite only cares about something like:

Is this request cross-site?

It doesn't understand: **This is a money transfer.**

A CSRF token lets the server require an application-validated value: `csrf_token=abc123`

The server can reject the request unless that value is present and valid.

Important: A CSRF token does not inherently prove user intent or need to be unique to every action. It provides an additional validation mechanism to distinguish legitimate application requests from requests an external attacker could forge.

A very important situation: subdomains

Suppose you have:

www.example.com
app.example.com
evil.example.com

Also, **SameSite is based on "site", not simply "origin."**

Modern browsers generally use the scheme and registrable domain when determining the site for SameSite purposes.

An origin is defined by:

- Scheme
- Hostname
- Port

For example:

https://app.example.com
https://www.example.com

are different **origins** but are the same **site**.

Likewise:

https://evil.example.com
https://app.example.com

are cross-origin but same-site.

However:

http://app.example.com
https://app.example.com

are considered cross-site under schemeful SameSite rules because their schemes differ.

Why this matters

If an attacker can compromise or control **evil.example.com**, SameSite may not provide the boundary you were expecting.

The attacker's page can initiate a request to **app.example.com**, and the browser may include the target application's session cookie because the request is considered same-site.

Important: This does not mean that **evil.example.com** must be able to read or steal the session cookie.

For example, a host-only cookie belonging to **app.example.com** is not automatically exposed to **evil.example.com**.

The browser may still attach that cookie when sending the request to **app.example.com**.

Cookie scope and SameSite classification are two different concepts.

A properly implemented CSRF token provides another independent check, provided the attacker cannot obtain, predict, or otherwise defeat its validation.

There's also a very important defense-in-depth reason

Think about these two configurations:

Application A

SameSite protection

Application B

SameSite protection + CSRF token

Application B has two different layers of protection.

If SameSite is accidentally weakened: **SameSite=None**

the CSRF token can still stop the attack.

If a developer accidentally removes CSRF-token validation: **SameSite=Strict**

may still stop many external cross-site attacks.

That's why real applications often use **multiple layers** instead of trusting one mechanism.

One more thing: SameSite isn't a replacement for authorization

This is extremely important for bug hunting.

Suppose you find: **POST /api/change-email**

and it requires: **Authorization: Bearer eyJ...**

The browser's automatic cookie behavior isn't the same security mechanism here.

The attacker generally can't simply make the victim's browser attach an arbitrary `Authorization` header containing the victim's bearer token.

That's a different situation from classic cookie-based CSRF.

However, this depends on how authentication is implemented.

If an application stores an access token in a cookie and relies on that automatically attached cookie for authentication, CSRF may still be relevant.

Likewise, some browser-managed authentication mechanisms can automatically supply credentials.

Don't classify an application as immune to CSRF merely because it uses tokens.

Determine **how those tokens are stored, transmitted, and attached to requests**.

Summary: SameSite vs. CSRF Tokens

SameSite

Controls:

> **"Should the browser send the cookie in this request, based on its site context?"**

SameSite → Browser-level protection → Controls cookie sending

CSRF token

Controls:

> **"Does this request contain a token that passes the application's CSRF validation?"**

CSRF token → Application-level protection → Server validates the request

And the direct answer to your question:

Yes, SameSite can eliminate many CSRF attacks. Modern browsers make it a strong defense.

But saying **"therefore CSRF tokens are unnecessary"** is too broad.

SameSite has limitations, depends on cookie configuration and site boundaries, and doesn't provide the same application-level validation that a CSRF token does.

For a serious web pentester, you should know **both**, because you'll encounter applications using:

- SameSite only
- CSRF tokens only
- Both

- Neither
- Origin/Referer or Fetch Metadata validation
- Custom-header-based protections
- Unusual combinations that are misconfigured

And those configurations produce very different CSRF testing results.

==========

Methodology for Discovery

1. Map State-Changing Requests

Use a proxy (Burp Suite/Caido) to identify requests that perform sensitive actions.

Pay particular attention to:

- **POST**
- **PUT**
- **PATCH**
- **DELETE**
- **GET** requests that improperly perform state-changing operations

Important: HTTP method alone does not determine whether an endpoint changes application state.

Some vulnerable applications allow sensitive actions through GET requests.

2. Identify the Authentication Mechanism

Determine how the application authenticates requests.

Check whether authentication relies on:

- Session cookies
- Other automatically attached browser credentials
- Bearer tokens explicitly added by client-side JavaScript
- Custom authentication headers

Key question:

> Can the victim's browser automatically supply the authentication credentials when an attacker induces a request?

If an attacker cannot cause the browser to supply the required credentials, a traditional CSRF attack may not be possible.

3. Check for CSRF Defenses

Check whether the application implements an effective defense against cross-site request forgery.

Examine:

- CSRF tokens
- Session cookie attributes
- SameSite restrictions
- Origin/Referer validation
- Fetch Metadata policies
- Custom-header requirements
- CORS behavior
- Re-authentication requirements

The absence of a CSRF token is an indicator for further investigation, **not proof of vulnerability**.

4. Test Token Validation

What happens if you remove the token?

- What happens if you provide a blank token?
- What happens if you provide an invalid token?
- What happens if you use a token from a different user's session?
- Does the endpoint behave differently if you change the request method?
- Does token validation depend on another cookie?
- Is the token actually validated against the appropriate session or cryptographic verification rules?

5. Examine Cookies

Check the **Set-Cookie** headers for attributes such as:

- **SameSite**
- **Secure**
- **HttpOnly**
- **Domain**
- **Path**

Determine whether the victim's browser would include the required authentication cookies in the proposed attack.

6. Examine Origin and Fetch Metadata Checks

Look for:

Origin: https://target.com

Referer: https://target.com/account

Sec-Fetch-Site: same-origin

Test whether the application correctly handles requests originating from another origin.

Important: Manually modifying these headers in Burp is useful for understanding server-side validation, but it does not mean an attacker can make a real browser send those same header values.

You must eventually verify browser-deliverable behavior.

7. Check Whether the Browser Can Generate the Required Request

This is one of the most important steps.

A manually modified request working in Burp Repeater does not automatically prove an exploitable CSRF vulnerability.

The attacker must be able to cause the victim's browser to send a request with the required:

- HTTP method
- Parameters
- Request body
- Content type
- Headers
- Authentication credentials

For example, ordinary HTML forms support these common content types:

- **application/x-www-form-urlencoded**
- **multipart/form-data**
- **text/plain**

A normal HTML form cannot directly send an arbitrary request with:

Content-Type: application/json

Cross-origin JavaScript may be able to construct a JSON request, but such requests normally require a successful CORS preflight.

If the preflight fails, the browser does not send the actual non-simple request.

Important distinction: A CORS error that prevents JavaScript from reading a response does not necessarily mean the request was never sent. Simple cross-origin requests can still reach the server and cause state changes even when the response is inaccessible.

8. Construct a Cross-Site Proof of Concept

Host the exploit on a different origin from the target application.

Use a browser with an authenticated test session to determine whether the forged request succeeds.

9. Verify the Actual State Change

Don't rely solely on **200 OK** or **403 Forbidden**

Verify whether the intended state-changing operation actually occurred.

For example, after attempting an email-change CSRF, check the account's stored email address using a separate legitimate request.

10. Confirm Real-World Exploitability

A valid CSRF finding generally requires demonstrating that:

1. The attacker can induce the request from an appropriate browser context.
2. The browser supplies the required authentication credentials.
3. Existing CSRF defenses do not stop the request.
4. The server performs the unintended action.

Test within your authorized scope and use controlled accounts for sensitive operations.

Remember:

> **A request that works in Burp Repeater is not necessarily a request that an attacker can reproduce from a malicious webpage.**

Detailed Explanation

In case of no CSRF Token Implementation

To craft a CSRF payload for a request that has no CSRF token implementation:

1. Intercept the particular state-changing request in Burp Suite.
2. Send it to Repeater and understand the required parameters.
3. Check the application's other CSRF defenses, especially SameSite, Origin/Referer validation, Fetch Metadata, and content-type restrictions.
4. If you have Burp Suite Professional, use **Engagement tools → Generate CSRF PoC**.
5. Alternatively, construct a local HTML proof of concept manually.
6. Test the payload in a real browser with an authenticated test session.
7. Verify that the sensitive operation actually succeeds.

Important: Avoid pasting raw authenticated requests containing session cookies, bearer tokens, or sensitive information into untrusted third-party CSRF generators.

Remember to modify action parameters only as required by the exploit.

You do not necessarily need to know or modify the victim's user ID. In traditional CSRF, the victim's authenticated session usually identifies the acting user.

If an arbitrary **uid** parameter independently determines whose account is modified, investigate whether a separate authorization vulnerability, such as IDOR, exists.

Example: CSRF Without a Token

Suppose the vulnerable endpoint is:

```
POST /change-email HTTP/1.1
Host: bank.com
Cookie: session=ABC123
Content-Type: application/x-www-form-urlencoded

email=attacker%40example.com
```

An attacker-controlled page could contain:

```
<!DOCTYPE html>
<html>
<body>

<form action="https://bank.com/change-email" method="POST">
  <input
    type="hidden"
    name="email"
    value="attacker@example.com"
  >
</form>

<script>
  document.forms[0].submit();
</script>

</body>
</html>
```

If the victim visits this page while authenticated, the browser may send the forged request.

The attack succeeds only if:

- The required authentication cookies or other credentials are included.
- The endpoint accepts the generated request format.
- The application has no effective CSRF protection blocking the operation.
- The submitted request satisfies any other applicable authorization and business-logic requirements.

In a successful traditional CSRF challenge, the web application trusts the authenticated browser request without adequately verifying whether the operation was initiated through an authorized application workflow.

How CSRF Tokens Stop This Attack

To stop this, developers can use a **CSRF Token (Anti-CSRF Token)**.

Using the synchronizer token pattern:

1. When a user logs in or the application initializes an appropriate session or protected form, the server generates a random, secret token, for example: **`c48ebdf9d2b587a11362083e8dc535bd`**.
2. The server embeds this secret token into the legitimate HTML form, such as the "Make Admin" page.
3. When the admin clicks submit, the browser sends the token back to the server: **`csrf_token=c48ebdf9d2b587a11362083e8dc535bd`**.
4. The server checks whether the submitted token matches the token associated with the current session.
5. The server rejects the state-changing operation if the token is missing or invalid.

Why this breaks your old script: Your malicious website normally cannot read the admin's secret token from the protected application because browsers prevent unrelated origins from reading protected cross-origin content through the Same-Origin Policy.

If you guess a token or leave it out, a correctly implemented server rejects the operation.

However, this assumes the token is:

- Unpredictable.
- Not leaked to the attacker.
- Properly validated.
- Bound to the appropriate session or securely verified under the selected token scheme.

A harder challenge may mean the developers *tried* to fix CSRF by adding a token, but made a coding mistake when checking it.

To solve such a challenge, investigate the implementation and identify whether one of the following weaknesses exists.

In case of CSRF Token Implementation

CSRF Token

A CSRF token is a secret, unpredictable value, or a securely constructed and validated token, used by a web application to protect sensitive operations against forged requests.

A typical synchronizer token is randomly generated and associated with the user's session. Other valid schemes may use cryptographically signed values instead of storing a separate token for every session or request.

Proper CSRF token validation helps prevent unauthorized commands from being executed on behalf of an authenticated user through traditional cross-site request forgery.

How It Works: Synchronizer Token Pattern

- The server creates a secure, random token when a user loads a protected form, logs in, or otherwise establishes an appropriate session.
- One copy of the token goes to the client browser, often inside a hidden form field or application response.
- Another copy is stored in the server-side session.
- The browser sends the token back when submitting a protected form or performing a sensitive action.
- The server compares the submitted token with its saved session token.
- The server accepts the request only if the token is valid and all other authentication, authorization, and business-logic requirements are satisfied.
- Otherwise, the server rejects the protected operation.

Why It Is Needed

- Attackers trick a user's browser into sending unwanted, forged requests to a trusted site where the user has an active session.
- Without an effective CSRF defense, the server may accept incoming authenticated requests merely because valid session credentials accompany them.
- Because an unrelated attacker-controlled origin ordinarily cannot obtain or predict a correctly protected CSRF token, forged requests without the required value fail.

Important CSRF Token Implementation Rules

A properly implemented CSRF token mechanism should:

- Use unpredictable values or cryptographically secure constructions.
- Validate tokens on the server.
- Require valid tokens for all relevant state-changing operations.
- Validate them consistently across supported methods and content types.
- Bind them to the correct authenticated session or securely validate that relationship.
- Avoid exposing tokens in URLs.

- Avoid leaking tokens through logs or untrusted external destinations.
- Avoid placing synchronizer tokens in cookies as a substitute for explicit request submission.
- Avoid relying on token presence, format, or length alone.
- Reject missing and invalid tokens consistently.

Important: CSRF tokens are not a replacement for authorization checks, and XSS can undermine them.

Common CSRF Bypasses to Test

To discover if the application's CSRF implementation is vulnerable, test the following scenarios by intercepting requests in Burp Suite/Caido or modifying your exploit script.

Use authorized test sessions and confirm real-browser exploitability before treating a bypass as a confirmed CSRF vulnerability.

Bypass Technique A: Remove the Token Entirely

Some poorly implemented validation routines only check the token if it is present in the request. If the parameter is missing, the backend script may skip validation entirely.

Sometimes, the developer's code effectively looks like this:

> "If a token is sent, make sure it is correct. If no token is sent, just skip the check."

Testing Method:

Remove the **csrf_token** parameter or header from the request completely and send it.

For example, change:

```
POST /change-email HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded

email=test@example.com&csrf_token=abc123
```

to:

```
POST /change-email HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded

email=test@example.com
```

If the application processes the sensitive operation when the required token is missing, the token validation is flawed.

Important: Verify that the same missing-token request can be generated from an attacker-controlled webpage with the necessary victim authentication.

Bypass Technique B: Send a Blank or Equal-Length Token

Similar to missing tokens, some flawed validation mechanisms only check whether a token exists or whether its length or format matches the expected standard, rather than validating its actual value.

Testing Method:

Change the token value to empty: `csrf_token=`

Or replace it with random characters of the same length.

For example:

Original: `csrf_token=a1b2c3d4`

Modified: `csrf_token=00000000`

Also test other invalid values where appropriate.

Important: An empty token or equal-length random token is not a universal bypass.

These tests look for flawed custom validation.

If the backend rejects the invalid values, that is expected behavior.

If an invalid value is accepted and the sensitive operation succeeds, investigate the validation logic and determine whether the request is exploitable through a real browser.

Bypass Technique C: Change the Request Method (POST to GET)

Some frameworks or custom code only enforce CSRF validation on specific HTTP methods like `POST`.

If the backend application accepts the parameters via `GET` as well, validation can sometimes be bypassed.

This may occur because the application uses method-independent parameter handling, such as PHP's `$_REQUEST` under configurations where it includes GET and POST variables.

Testing Method:

Change the request method from `POST` to `GET` and append the parameters to the URL query string.

For example:

Original:

```
POST /admin/upgrade HTTP/1.1
Host: target.com
Content-Type: application/x-www-form-urlencoded

user_id=4&csrf_token=abc123
```

Modified:

```
GET /admin/upgrade?user_id=4 HTTP/1.1
Host: target.com
```

If the application performs the sensitive action without requiring the token, the method-dependent validation may be vulnerable.

Important SameSite consideration

With `SameSite=Lax`, a cross-site **top-level GET navigation** may include the victim's session cookie.

For example:

```
<script>
   window.location =
      "https://target.com/admin/upgrade?user_id=4";
</script>
```

If the endpoint accepts the GET request and performs the action, the attack may succeed.

However, using an `<img>` tag is not equivalent to a top-level navigation.

For example:

```
<img src="https://target.com/admin/upgrade?user_id=4">
```

An image request is a subresource request, so a `SameSite=Lax` session cookie is not normally included when it originates cross-site.

Remember: Method changes must be verified in a browser, not only in Burp Repeater.

Bypass Technique D: Use a Token from a Different Session

A common flaw occurs when the application verifies that the token is valid within the system but fails to verify that the token belongs specifically to the *currently logged-in session*.

Testing Method:

1. Log into the application using Account A.
2. Generate a valid CSRF token.
3. Log into Account B in a separate session.
4. Replace Account B's CSRF token with Account A's token.

5. Submit the protected action while authenticated as Account B.

If the application accepts Account A's token under Account B's session, the token is not properly session-bound.

For example:

Token generated for Account A: `csrf_token=TOKEN_A`

Request authenticated as Account B:

```
POST /change-email HTTP/1.1
Host: target.com
Cookie: session=SESSION_B
Content-Type: application/x-www-form-urlencoded

email=test@example.com&csrf_token=TOKEN_A
```

If the operation succeeds, investigate whether an attacker could obtain their own valid token and place it in a forged browser request targeting a victim.

Important: The attacker's token being accepted in another session is a serious validation flaw, but practical CSRF still depends on whether the victim's browser can deliver the forged request.

Bypass Technique E: Double-Submit Cookie Flaws

Some applications use a **Double-Submit Cookie** defense, where a CSRF value is provided in a cookie and separately in a request parameter or custom header.

The server checks whether the submitted value corresponds to the expected cookie/token relationship.

Naive Double-Submit Cookie

In a naive implementation, the server may simply compare:

Cookie: `csrf=test`

against:

```
csrf=test
```

If the values match, the server accepts the request.

This implementation can be vulnerable if an attacker can manipulate the CSRF cookie in the victim's browser.

Testing Method:

Check whether changing both the CSRF cookie and the request parameter to the same arbitrary value allows the request to pass.

For example:

```
POST /change-email HTTP/1.1
Host: target.com
Cookie: session=SESSION123; csrf=test
Content-Type: application/x-www-form-urlencoded

email=test@example.com&csrf=test
```

If this works in Burp Repeater, it suggests the server may rely on naive equality validation.

But this alone does not prove exploitable CSRF.

An attacker running JavaScript on **evil.com** cannot simply set an arbitrary Cookie request header for **target.com**.

You must also determine whether the attacker can realistically plant or manipulate the required CSRF cookie in the victim's browser.

Possible cookie-injection primitives can involve:

- An attacker-controlled sibling subdomain.
- A suitable subdomain takeover.
- A vulnerable cookie-setting mechanism.
- Another application weakness that permits cookie injection.

The exact exploitability depends on cookie scope, prefixes, Secure restrictions, and the target's parsing behavior.

Secure Alternative: Signed Double-Submit Cookie

A properly implemented signed double-submit cookie uses cryptographic validation and explicitly binds the token to the authenticated session.

For example, the application can use an HMAC derived from a server-side secret and appropriate session-specific data.

The server verifies the signature and session binding rather than merely checking whether two attacker-controlled strings match.

Important: Double-submit cookies are not inherently insecure. The weakness lies in naive or improperly implemented validation.

Technique F: HTTP Method Override

Some applications, frameworks, or proxies support headers such as: **X-HTTP-Method-Override** to allow a request using one HTTP method to be treated by the backend as another method.

For example, the application may receive a `GET` request but interpret it as a `POST` because of a supported override mechanism.

Testing Method:

If CSRF protection is enforced for **POST** but the application supports method overriding, test whether a request using an alternative method together with an appropriate method-override mechanism is processed as the protected method.

For example:

GET /change-email?email=attacker@example.com HTTP/1.1
Host: target.com
X-HTTP-Method-Override: POST

If the application processes this as a POST without requiring the appropriate CSRF protection, the method-override mechanism may introduce a vulnerability.

Critical browser limitation

A normal HTML form or link cannot attach arbitrary custom HTTP headers such as:

X-HTTP-Method-Override: POST

A cross-origin JavaScript request with this custom header generally requires a successful CORS preflight.

Therefore, a header-based method override working in Burp Repeater does not automatically establish exploitable CSRF.

Parameter-based method override

Some frameworks also support method overriding through request parameters.

For example:

GET /change-email?email=attacker@example.com&_method=POST HTTP/1.1

Host: target.com

If the target supports this mechanism, a browser may be able to send the request through a top-level navigation.

Under applicable conditions, this may bypass CSRF validation that checks the original method while routing interprets the overridden method.

Important: The **_method** parameter is framework-dependent. It is not universally supported.

Always verify the actual method-override implementation and the browser's ability to reproduce the request.

Technique G: CSRF Token Tied to a Non-Session Cookie

Some applications do not bind CSRF tokens to the authenticated session.

Instead, they bind a CSRF token to a separate cookie.

For example:

```
Cookie: session=SESSION123; csrfKey=KEY123
```

The application may validate the submitted CSRF token against **csrfKey**, while the authenticated session is determined independently by **session**.

This can introduce a vulnerability if the attacker can obtain a valid token/cookie pair and make the victim's browser use the attacker's CSRF cookie.

Testing Method:

1. Identify the authenticated session cookie.
2. Identify any separate cookie involved in CSRF validation.
3. Determine whether changing the CSRF-related cookie affects token validation without changing the authenticated user.
4. Using two test accounts, determine whether a valid token and related CSRF cookie from one session can be accepted alongside another session's authentication cookie.
5. Investigate whether a realistic cookie-injection primitive exists.

Important: Merely changing the cookie manually in Burp does not prove exploitability.

An attacker must be able to reproduce the required cookie state in the victim's browser.

Technique H: Bypassing Flawed Origin or Referer Validation

Some applications rely on the **Origin** or **Referer** header to decide whether a request is legitimate.

This can be effective when correctly implemented, but flawed validation may introduce bypasses.

Example: Validation only when a header is present

A poorly implemented backend may behave like this:

> "If the Referer header is present, validate it. If it is missing, allow the request."

An attacker may be able to trigger browser requests where the relevant header is absent, depending on the request context and browser policies.

Example: Weak hostname validation

Suppose the legitimate application is: **https://example.com**

An insecure origin check might incorrectly accept: **https://example.com.attacker.com**

because the developer uses an unsafe string-prefix comparison instead of validating the actual origin.

Testing Method:

- Check whether invalid origins are rejected.
- Check how the application handles missing Origin or Referer headers.
- Check whether **Origin: null** is improperly trusted.
- Check whether the application incorrectly accepts attacker-controlled domains containing the legitimate domain name.
- Check whether redirects or supported browser contexts affect the validation outcome.

Important: Browsers control these headers. An attacker generally cannot arbitrarily set a forged Origin or Referer header using normal webpage JavaScript.

A bypass is only a confirmed practical CSRF issue when the necessary browser-generated request can actually be produced.

Technique I: Bypassing SameSite Cookie Restrictions

SameSite is a strong defense against many traditional CSRF attacks, but it has limitations.

1. Bypassing Lax Restrictions Using GET Requests

A cross-site top-level navigation using GET may include cookies configured with **SameSite=Lax**.

If a sensitive endpoint performs state-changing operations through GET, an attacker may exploit it.

Example:

```
<script>
   window.location =
      "https://target.com/change-email?email=attacker@example.com";
</script>
```

This only works if the endpoint accepts the request, includes the necessary authentication, and lacks an effective independent CSRF defense.

2. Exploiting Same-Site Sibling Domains

Suppose the target application is: **https://app.example.com**

And the attacker controls: **https://evil.example.com**

These are different origins but the same site.

SameSite restrictions generally do not block cookies merely because the request comes from a different origin within the same site.

This can enable CSRF if the attacker controls a sibling domain and the target relies only on SameSite without effective additional defenses.

3. Using On-Site Gadgets

Some applications contain client-side functionality that can trigger another request within the same site based on attacker-controlled input.

For example, a vulnerable client-side redirect may allow an attacker to cause the browser to navigate from a legitimate page to a sensitive endpoint.

In some situations, the secondary request is treated as same-site and includes cookies that would have been excluded from the initial cross-site request.

This is different from assuming every server-side redirect bypasses SameSite.

The browser's redirect handling and request context matter.

4. Recently Issued Cookies Under Lax-by-Default

Some browsers apply more permissive behavior when a cookie lacks an explicit SameSite attribute.

Under that behavior, a recently set cookie may be included in a cross-site top-level POST request for approximately two minutes.

This can sometimes be relevant where an attacker can cause a new authentication cookie to be issued or refreshed.

Important: This is a browser-specific behavior that must be tested. It is not a universal bypass of explicitly configured **SameSite=Lax**.

Technique J: Client-Side CSRF

Traditional CSRF involves an attacker-controlled page directly generating a request to the target application.

Client-side CSRF is different.

In client-side CSRF, the attacker manipulates input that the legitimate application's own JavaScript uses when constructing a sensitive request.

The application may unknowingly make the forged request itself.

Example

Suppose the legitimate application contains JavaScript that reads an endpoint from the URL fragment:

```
const endpoint = location.hash.slice(1);

fetch(endpoint, {
    method: "POST",
    headers: {
        "X-CSRF-Token": csrfToken
    },
    credentials: "same-origin"
});
```

This is a simplified example. Whether it is exploitable depends on the endpoint, request body, attacker control, and application behavior.

If an attacker can influence the endpoint in a way that causes a sensitive same-origin action, the application's own JavaScript may attach:

- The user's session credentials.
- A valid CSRF token.
- Trusted application headers.

The result may be an unauthorized state-changing request.

Why this matters

The attacker does not necessarily need to read or steal the CSRF token.

The legitimate JavaScript may automatically include it.

SameSite restrictions also do not solve this problem when the sensitive request is generated as a same-origin request by the legitimate application.

Testing Method:

Identify application JavaScript that constructs requests using attacker-controllable values such as:

- URL fragments.
- Query parameters.
- `postMessage` data.
- Other client-side input.

Determine whether an attacker can influence the destination, method, parameters, or body of a sensitive request.

Verify whether legitimate application JavaScript uses those values to perform an unauthorized action.

Important: Not every instance of attacker-controlled input reaching `fetch()` is client-side CSRF. You must establish an actual security impact and confirm that the generated request performs an unauthorized action.

Additional CSRF Variant: Login CSRF

Traditional CSRF commonly targets users who are already authenticated.

Login CSRF can target the login process itself.

Example

An attacker creates or controls an account on a website.

The attacker then causes a victim's browser to submit a login request using the attacker's credentials.

If the application accepts that request, the victim's browser may become authenticated to the attacker's account.

The victim may then unknowingly perform actions or enter sensitive information into an account controlled by the attacker.

Why this matters

The victim does not need to be logged into the target application before the attack.

Important: Login CSRF does not automatically establish account takeover of the victim's existing account. Its impact depends on how the application handles sessions and what information or actions become associated with the attacker-controlled account.

==========

Final CSRF Testing Checklist

When testing an application for CSRF, follow this sequence:

1. **Identify a sensitive operation.** Find an endpoint that changes application state or performs a meaningful action under a user's authority.
2. **Determine how authentication works.** Find out whether the browser automatically supplies the required credentials.
3. **Inspect existing CSRF defenses.** Look for CSRF tokens, SameSite restrictions, Origin/Referer validation, Fetch Metadata, and custom-header requirements.
4. **Test token validation.** Try missing, empty, invalid, and cross-session tokens where appropriate.
5. **Test alternative HTTP methods.** Check GET-based state changes and supported method-override mechanisms.
6. **Investigate cookie-based validation.** Determine whether CSRF cookies can be manipulated and whether tokens are bound to the correct authenticated session.

7. **Examine same-site attack surfaces.** Consider sibling subdomains, on-site gadgets, and client-side request construction.
8. **Understand browser restrictions.** Check whether the required method, content type, headers, cookies, and request body can be reproduced from an attacker-controlled origin.
9. **Create a cross-site proof of concept.** Use a controlled attacker-origin page and a browser with an authenticated test session.
10. **Verify the result.** Confirm that the operation actually succeeded without legitimate user authorization or intent.
11. **Evaluate impact.** Determine what an attacker could realistically accomplish, avoiding assumptions such as treating every forced email change as automatic account takeover.
12. **Document the evidence.** Record the vulnerable endpoint, required conditions, defenses bypassed, browser behavior, and actual state change.

==========

The Most Important Things to Remember

1. CSRF is not simply sending a cross-origin request.

A CSRF attack must cause the application to perform an unwanted action under the victim's authority.

2. Browsers do not automatically attach all cookies to every request.

Cookie attributes and browser security policies determine whether cookies are included.

3. The absence of a CSRF token does not automatically mean the endpoint is vulnerable.

The application may have other effective protections.

4. The presence of a CSRF token does not automatically mean the endpoint is protected.

The token must be properly generated, transmitted, and validated.

5. SameSite and Same-Origin Policy are different concepts.

A request can be cross-origin but same-site.

6. SameSite=Lax permits cookies on qualifying cross-site top-level safe-method navigations, not arbitrary cross-site requests.

This is why GET-based state-changing actions are especially relevant.

7. A successful request in Burp Repeater does not prove exploitable CSRF.

The attacker must be able to reproduce the request through the victim's browser.

8. CORS restricting response access does not automatically prevent CSRF.

Ordinary cross-origin form submissions can reach servers without CORS permission.

9. A correctly implemented CSRF token must not be predictable, exposed, or improperly validated.

Session binding or equivalent cryptographically sound verification is critical.

10. XSS can undermine CSRF protections.

An attacker executing JavaScript within the target application's security context can often use the application's own credentials and tokens.

11. A valid CSRF token does not establish authorization.

Authentication, authorization, and CSRF protection are separate security controls.

12. The final proof is the actual unauthorized operation.

Do not judge a CSRF vulnerability solely by HTTP status codes, missing tokens, or modified requests that work in a proxy.

==========

References

1. OWASP Foundation — Cross-Site Request Forgery Prevention Cheat Sheet
2. PortSwigger Web Security Academy — Cross-Site Request Forgery (CSRF)
3. PortSwigger Web Security Academy — Bypassing CSRF Token Validation
4. PortSwigger Web Security Academy — Bypassing SameSite Cookie Restrictions
5. PortSwigger Web Security Academy — How to Prevent CSRF Vulnerabilities
6. PortSwigger — Generate CSRF PoC
7. MDN Web Docs — Set-Cookie HTTP Header
8. MDN Web Docs — Cross-Origin Resource Sharing (CORS)

==========